

MỤC LỤC

MỤC LỤC.....	1
HƯỚNG DẪN	3
BÀI 1: LÀM QUEN VỚI NGÔN NGỮ C.....	1
1.1 TÓM TẮT LÝ THUYẾT	1
1.1.1 Các ký hiệu	1
1.1.2 Các kiểu dữ liệu cơ bản trong C.....	2
1.1.3 Bảng ký hiệu và phép toán	2
1.1.4 Lệnh nhập, xuất dữ liệu cơ bản	3
1.2 THỰC HÀNH CƠ BẢN	3
1.3 THỰC HÀNH NÂNG CAO.....	6
1.4 BÀI TẬP VỀ NHÀ.....	7
BÀI 2: CẤU TRÚC Rẽ NHÁNH	8
2.1 TÓM TẮT LÝ THUYẾT	8
2.1.1 Cấu trúc if.....	8
2.1.2 Cấu trúc if ...else	8
2.1.3 Cấu trúc switch	9
2.2 THỰC HÀNH CƠ BẢN	9
2.3 THỰC HÀNH NÂNG CAO.....	14
2.4 BÀI TẬP VỀ NHÀ.....	14
BÀI 3: CẤU TRÚC LẶP.....	16
3.1 TÓM TẮT LÝ THUYẾT	16
3.1.1 Cấu trúc for.....	16
3.1.2 Cấu trúc while.....	16
3.1.3 Cấu trúc do ... while.....	17
3.2 THỰC HÀNH CƠ BẢN	17
3.3 THỰC HÀNH NÂNG CAO.....	22
3.4 BÀI TẬP VỀ NHÀ.....	23
BÀI 4: HÀM	24
4.1 TÓM TẮT LÝ THUYẾT	24
4.1.1 Cấu trúc một chương trình C theo hàm	24
4.1.2 Cách xây dựng một hàm con	25
4.1.3 Tham số của hàm.....	26
4.1.4 Cách gọi hàm con ra thực hiện.....	26
4.2 THỰC HÀNH CƠ BẢN	27
4.3 THỰC HÀNH NÂNG CAO.....	30
4.4 BÀI TẬP VỀ NHÀ.....	30
BÀI 5: MẢNG MỘT CHIỀU	32

5.1 TÓM TẮT LÝ THUYẾT	32
5.2 THỰC HÀNH CƠ BẢN	33
5.3 THỰC HÀNH NÂNG CAO.....	35
5.4 <i>BÀI TẬP VỀ NHÀ</i>	37
BÀI 6: MẢNG HAI CHIỀU	39
6.1 TÓM TẮT LÝ THUYẾT	39
6.2 THỰC HÀNH CƠ BẢN	40
6.3 THỰC HÀNH NÂNG CAO.....	43
6.4 <i>BÀI TẬP VỀ NHÀ</i>	43
BÀI 7: CHUỖI KÝ TỰ.....	45
7.1 TÓM TẮT LÝ THUYẾT	45
7.2 THỰC HÀNH CƠ BẢN	46
7.3 THỰC HÀNH NÂNG CAO.....	48
7.4 <i>BÀI TẬP VỀ NHÀ</i>	49
TÀI LIỆU THAM KHẢO	50

HƯỚNG DẪN

Học phần Thực hành Cơ sở Lập trình cung cấp cho sinh viên những kỹ năng thực hành cơ bản về ngôn ngữ lập trình C. Học phần này là nền tảng để tiếp thu hầu hết các học phần khác trong chương trình đào tạo. Mặt khác, nắm vững học phần này là cơ sở để phát triển tư duy và kỹ năng lập trình để giải các bài toán và các ứng dụng trong thực tế.

Học xong học phần này, sinh viên phải nắm được các vấn đề sau:

- Tổng quan về Ngôn ngữ lập trình C.
- Các kiểu dữ liệu trong C.
- Các lệnh có cấu trúc.
- Cách thiết kế và sử dụng các hàm trong C.
- Xử lý các bài toán trên mảng một chiều
- Xử lý các bài toán trên mảng hai chiều.

NỘI DUNG HỌC PHẦN

- Bài 1: Làm quen với Ngôn ngữ C.
- Bài 2: Cấu trúc rẽ nhánh.
- Bài 3: Cấu trúc lặp.
- Bài 4: Hàm
- Bài 5: Mảng một chiều
- Bài 6: Mảng hai chiều
- Bài 7: Chuỗi ký tự

YÊU CẦU HỌC PHẦN

Có tư duy tốt về logic và kiến thức toán học cơ bản.

CÁCH TIẾP NHẬN NỘI DUNG HỌC PHẦN

Thực hành Cơ sở Lập trình là học phần đầu tiên giúp sinh viên làm quen với phương pháp lập trình trên máy tính, giúp sinh viên có khái niệm cơ bản về cách tiếp cận và giải quyết các bài toán tin học, giúp sinh viên có khả năng tiếp cận với cách tư duy của người lập trình, và là tiền đề để tiếp cận với các học phần quan trọng còn lại của ngành Công nghệ Thông tin. Vì vậy, yêu cầu người học phải tham dự đầy đủ các buổi học thực hành tại lớp, thực hành lại các bài tập ở nhà, nghiên cứu tài liệu trước khi đến lớp và gạch chân những vấn đề không hiểu khi đọc tài liệu để đến lớp trao đổi.

PHƯƠNG PHÁP ĐÁNH GIÁ HỌC PHẦN

Để học tốt học phần này, người học cần ôn tập các bài đã học, trả lời các câu hỏi và làm đầy đủ bài tập; đọc trước bài mới và tìm thêm các thông tin liên quan đến bài học.

Đối với mỗi bài học, người học đọc trước phần tóm tắt kiến thức lý thuyết, sau đó thực hành các bài từ cơ bản đến nâng cao. Lưu ý xem kỹ hướng dẫn trong mỗi bài thực hành và làm theo. Kết thúc mỗi bài học, học viên cần làm lại các bài thực hành ở nhà và luyện tập thêm.

Học phần được đánh giá gồm:

1. **Điểm quá trình** (30%) gồm chuyên cần và điểm tích cực.
2. **Điểm thực hành** (70%) là trung bình điểm các buổi thực hành.

Điểm học phần = Điểm quá trình + Điểm thực hành.

BÀI 1: LÀM QUEN VỚI NGÔN NGỮ C

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Làm quen với cấu trúc chung của một chương trình C đơn giản.
- Các lệnh nhập, xuất dữ liệu (*scanf, printf*).
- Các kiểu dữ liệu chuẩn (*int, long, char, float, ...*).
- Các phép toán và các hàm chuẩn của ngôn ngữ lập trình C.
- Thực hiện viết các chương trình hoàn chỉnh sử dụng các lệnh đơn giản.

1.1 TÓM TẮT LÝ THUYẾT

1.1.1 Các ký hiệu

Ký hiệu	Diễn giải	Ví dụ
{ }	Bắt đầu và kết thúc hàm hay khối lệnh.	<pre>int main() { }</pre>
;	Kết thúc khai báo biến, một lệnh, một lời gọi hàm, hay khai báo nguyên mẫu hàm.	<pre>int x; void NhapMang(int a[], int &n);</pre>
//	Chú thích (ghi chú) cho một dòng. Chỉ có tác dụng đối với người đọc chương trình.	<pre>// Ham nay dung de nhap mang void NhapMang(int a[], int &n);</pre>
/* */	Chú thích cho trường hợp nhiều dòng. Chỉ có tác dụng với người đọc chương trình.	<pre>/* Dau tien nhap vao n. Sau do nhap cho tung phan tu */ void NhapMang(int a[], int &n);</pre>

1.1.2 Các kiểu dữ liệu cơ bản trong C

Kiểu	Ghi chú	Kích thước	Định dạng
Kiểu liên tục (số thực)			
float	Số thực độ chính xác đơn	4 bytes	%f
double	Số thực độ chính xác kép	8 bytes	%f
long double	Số thực độ chính xác gấp 4	10 bytes	%Lf hoặc %lf
Kiểu rời rạc (số nguyên)			
char	Ký tự (hoặc)	1 byte	%c
	Số nguyên	1 byte	%d
unsigned char	Số nguyên dương	1 byte	%d
int	Số nguyên	2 bytes	%d
unsigned int	Số nguyên dương	2 bytes	%u
long	Số nguyên	4 bytes	%ld
unsigned long	Số nguyên dương	4 bytes	%lu

1.1.3 Bảng ký hiệu và phép toán

Phép toán	Ý nghĩa	Ghi chú
+, -, *, / %	Cộng, trừ, nhân, chia Chia lấy phần dư	Ví dụ: 5%2=1
>, >=, <, <=, == !=	Lớn hơn, lớn hơn hoặc bằng, nhỏ hơn, nhỏ hơn hoặc bằng Bằng nhau Khác nhau	
! && 	NOT AND OR	
++ --	Tăng 1 Giảm 1	Nếu toán tử tăng giảm đặt trước thì tăng giảm trước rồi tính biểu thức hoặc ngược lại.

1.1.4 Lệnh nhập, xuất dữ liệu cơ bản

Cú pháp	Diễn giải	Ví dụ
<code>scanf("mã định dạng", địa chỉ của các biến);</code>	Nhập dữ liệu: lấy dữ liệu nhập từ bàn phím lưu vào biến.	<code>int x; float y; scanf("%d%f", &x, &y);</code>
<code>printf("chuỗi định dạng", các biểu thức);</code>	Xuất 1 thông báo ra màn hình. Xuất giá trị của các biểu thức lên màn hình.	<code>printf("Day la chuong trinh vi du minh hoa\n"); printf("Gia tri cua so vua nhap la %d", x);</code>

1.2 THỰC HÀNH CƠ BẢN

Câu 1: Làm quen với cấu trúc chung của một chương trình C đơn giản:

Mở C-free/Dev-C, vào File/New/Source file.

Gõ đoạn code sau và lưu file với phần mở rộng là .cpp (Ví dụ: Cau1.cpp)

```
// Khai báo thư viện stdio.h vì nó chứa hàm printf.
#include <stdio.h>
int main()
{
    printf("Hello, World!");
    printf("Hello, World!\n");
    printf("Hello, \nWorld!\n");
    printf("Hello, \tWorld!\n");
    return 0;
}
```

Hãy biên dịch và chạy chương trình, xem kết quả trên màn hình và rút ra nhận xét.

Phím tắt để biên dịch và chạy chương trình (Compile and Run):

- C-free: F5
- Dev-C: F11

Câu 2: Viết chương trình in lên màn hình một thiệp mời dự sinh nhật có dạng:

THIỆP MỜI

Thân moi ban: "Le Loi"

Toi du le sinh nhat cua minh

Vao luc 19h ngay 20/10/2016

Tai: 05/42 Vinh Vien – TP. HCM

Rat mong duoc don tiep!

Ho Le Thu

- Hướng dẫn: Áp dụng \t (tab), \n (xuống dòng), \" (in ra dấu ")

Câu 3: Viết chương trình thực hiện:

- Nhập một số nguyên. Xuất ra màn hình số nguyên vừa nhập.
- Nhập một số thực. Xuất ra màn hình số thực vừa nhập.
- Nhập một kí tự. Xuất ra màn hình kí tự vừa nhập.
- Nhập hai số nguyên. Hãy tính tổng, hiệu, tích, thương của hai số đó và xuất kết quả ra màn hình.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    //câu a
```

```
    //Bước 1: khai báo số nguyên a
```

```
    int a;
```

```
    printf("Moi nhap 1 so nguyen: ");
```

```
    //Bước 2: nhập số nguyên a
```

```
    scanf("%d", &a);
```

```
    //Bước 3: kiểm tra giá trị số nguyên a vừa nhập đúng hay không
```

```
    printf("So nguyen da nhap la %d", a);
```

```
    //câu b tương tự
```

```
    float b ;
```

```
    printf("Moi nhap 1 so thuc: " );
```

```
    scanf("%f ", &b);
```



```

printf("So thuc da nhap la %f", b);
//câu c
char z;
printf("Moi nhap 1 kí tự: ");
fflush(stdin); //xoá bộ nhớ đệm
scanf("%c", &z);
printf("Ki tu da nhap la %c", z);
//câu d
int x, y, z;
printf("Moi nhap hai so nguyen de cong: ");
scanf("%d%d", &x, &y);
z = x + y;
printf("Tong hai so la %d\n", z);
//hoặc printf("Tong hai so la %d\n", x+y);
/* tương tự với phép hiệu, tích, thương. Lưu ý kiểu dữ liệu
kết quả của phép chia */
return 0;
}

```

Câu 4: Viết chương trình nhập vào bán kính r của một hình tròn. Tính chu vi và diện tích của hình tròn. In các kết quả lên màn hình.

Hướng dẫn:

Khai báo biến r để lưu trữ bán kính của hình tròn

Khai báo biến: **Kiểu_dữ_liệu Tên_biến;**

VD: float r ;

Khai báo hằng số $PI = 3.14$

Cách 1: #define Tên_hằng Giá_trị.

Gợi ý: nên đặt câu lệnh trước hàm main

VD: #define MAX 100

Cách 2: const Kiểu_dữ_liệu Tên_hằng=Giá trị;

Gợi ý: nên đặt câu lệnh trong hàm main

VD: const int MAX=100;

Diện tích hình tròn: $\text{PI} * r * r$

Chu vi hình tròn: $2 * \text{PI} * r$.

Câu 5: Viết chương trình chuyển đổi nhiệt độ từ độ F sang độ C và ngược lại, biết rằng:

Đổi từ	Sang	Công thức
Fahrenheit	Celsius	$^{\circ}\text{C} = 5/9(\text{F}-32)$
Celsius	Fahrenheit	$^{\circ}\text{F} = 9/5 \text{ C} + 32$

1.3 THỰC HÀNH NÂNG CAO

Câu 6: Viết chương trình đảo ngược một số nguyên dương có đúng 3 chữ số.

VD: Nhập vào $n=234 \rightarrow$ In ra: 432

Hướng dẫn:

- Lần lượt lấy các chữ số (sử dụng phép chia lấy phần nguyên '/' và phép chia lấy phần dư '%') và in ra màn hình theo thứ tự:
 - Chữ số hàng đơn vị
 - Chữ số hàng chục
 - Chữ số hàng trăm.

Ví dụ: $234 \% 10 = 4$

$234 / 10 = 23$

$23 \% 10 = 3$

$23 / 10 = 2$

In theo thứ tự ngược lại 432 hoặc sử dụng phép nhân và phép cộng để chuyển các chữ số thành một số có ba chữ số như 432.

Câu 7: Cho 2 số nguyên a và b, hãy viết chương trình để hoán đổi hai giá trị a và b. Xuất ra kết quả hai giá trị a và b trước khi hoán vị và sau khi hoán vị.

Hoán vị hai giá trị a và b với các điều kiện:

- Hoán vị a và b sử dụng biến thứ 3 để lưu trữ tạm

- b. Hoán vị a và b sử dụng toán tử '+' và '-'
- c. Hoán vị a và b sử dụng toán tử '*' và '/'
- d. Hoán vị a và b sử dụng toán tử XOR.

1.4 BÀI TẬP VỀ NHÀ

Câu 8: Cho tam giác ABC có độ dài cạnh đáy x (cm) và chiều cao h (cm), hãy viết chương trình tính diện tích của tam giác ABC. Biết rằng: công thức tính diện tích tam giác thường $S = (a * h)/2$ (cm²).

Câu 9: Viết chương trình chuyển một số nguyên dương có đúng 5 chữ số thành một số nguyên dương có đúng 3 chữ số. Biết rằng: số có 3 chữ số là số đã loại bỏ biên đầu và biên cuối của số có 5 chữ số.

Câu 10: Viết chương trình tính $S(n) = x + x^3 + x^5 + x^7$. Biết rằng: x là số thực nhập từ bàn phím.

BÀI 2: CẤU TRÚC RẼ NHÁNH

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Nắm vững cú pháp, cách thức hoạt động của cấu trúc rẽ nhánh *if* và *if ... else*.
- Nắm vững cú pháp, cách thức hoạt động của cấu trúc *switch ... case*

2.1 TÓM TẮT LÝ THUYẾT

2.1.1 Cấu trúc *if*

Cú pháp:

```
if (biểu thức điều kiện)
    công việc;
```

Ý nghĩa: Nếu biểu thức điều kiện cho kết quả đúng (khác không) thì thực hiện công việc.

2.1.2 Cấu trúc *if ...else*

Cú pháp:

```
if (biểu thức điều kiện)
    công việc 1;
else
    công việc 2;
```

Ý nghĩa: Nếu biểu thức điều kiện cho kết quả đúng thì thực hiện công việc 1, ngược lại thì thực hiện công việc thứ 2.

❖ **Lưu ý:**

- Biểu thức điều kiện phải đặt trong cặp dấu ngoặc ().
- Công việc cần thực hiện có thể gồm 1 hay nhiều lệnh. Nếu gồm nhiều lệnh con thì các lệnh con phải được gom vào trong cặp dấu {} → gọi là khối lệnh.

2.1.3 Cấu trúc switch

Cú pháp:

```
switch (bieu_thuc)
{
    case gia_tri_i:
        các câu lệnh;
        break; /* lệnh break giúp chương trình thoát khỏi lệnh case sau
                khi thực hiện xong một trường hợp*/
    ...
    case gia_tri_n:
        các câu lệnh;
        break;
    [default: công việc thực hiện mặc định nếu biểu thức không có giá
    trị nào trong các case] //có thể có hoặc không tùy bài toán
}
```

Gia_tri_i: là các hằng số nguyên hoặc ký tự.

Phụ thuộc vào giá trị của biểu thức viết sau switch, nếu:

- Giá trị này = *gia_tri_i* thì thực hiện câu lệnh sau case *gia_tri_i*.
- Khi giá trị biểu thức không thỏa tất cả các *gia_tri_i* thì thực hiện câu lệnh sau default nếu có, hoặc thoát khỏi câu lệnh switch.
- Khi chương trình đã thực hiện xong câu lệnh của case *gia_tri_i* nào đó thì nó sẽ thực hiện luôn các lệnh thuộc case bên dưới nó mà không xét lại điều kiện (do các *gia_tri_i* được xem như các nhãn). Vì vậy, để chương trình thoát khỏi lệnh case sau khi thực hiện xong một trường hợp, ta dùng lệnh **break**.

2.2 THỰC HÀNH CƠ BẢN

Câu 1: Viết chương trình thực hiện:

- Nhập vào hai số nguyên. Xuất ra màn hình giá trị lớn nhất
- Nhập vào ba số nguyên. Xuất ra màn hình giá trị lớn nhất

Hướng dẫn:

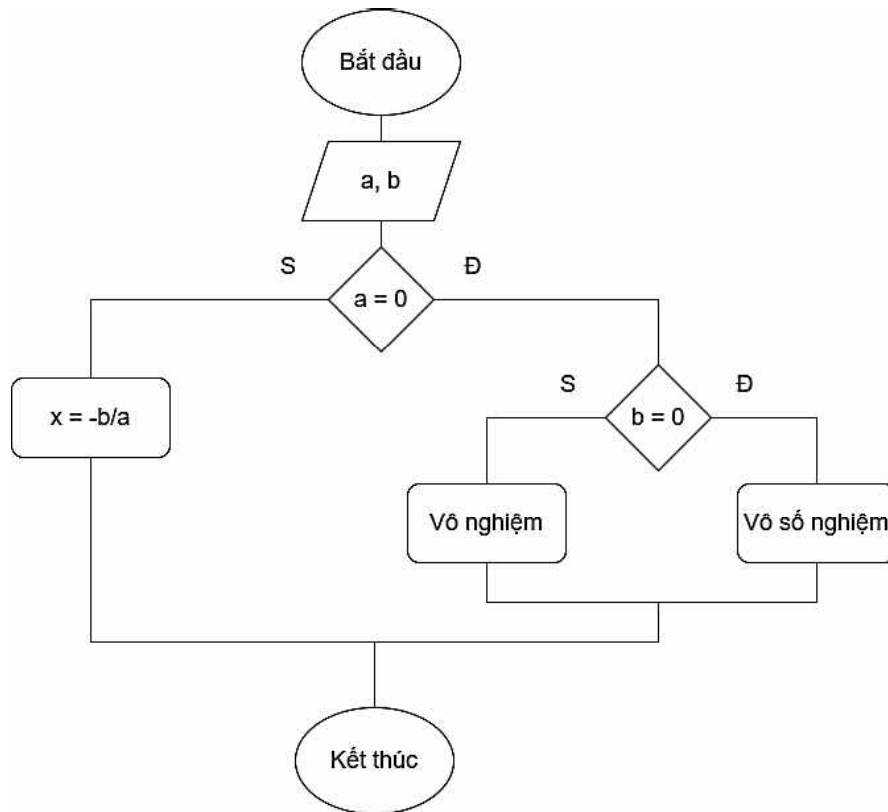
- ❖ Nhập vào hai số nguyên. Xuất ra màn hình giá trị lớn nhất
- Khai báo hai biến *a*, *b* để lưu hai số nguyên

- Thông báo và cho người dùng nhập 2 số nguyên a và b (dùng printf để thông báo, scanf để nhập tuần tự 2 biến số nguyên)
- Sử dụng một biến max để lưu giá trị lớn nhất trong hai số, khai báo max
- Gán max bằng giá trị của biến a
- So sánh max và b , nếu b lớn hơn thì gán $max = b$
- Xuất ra màn hình giá trị lớn nhất tìm được: xuất giá trị biến max
- ❖ Nhập vào ba số nguyên. Xuất ra màn hình giá trị lớn nhất
 - Tương tự, khai báo 3 biến a, b, c và max lưu giá trị lớn nhất
 - Giả sử a là số lớn nhất, tức $max = a$
 - Lần lượt so sánh hai số còn lại b, c với max , nếu số nào lớn hơn max thì cập nhật $max =$ số đó, như sau: Nếu $b > max$ thì $max = b$. Nếu $c > max$ thì $max = c$
 - Xuất ra màn hình giá trị lớn nhất tìm được: xuất giá trị biến max .

Câu 2: Viết chương trình giải phương trình bậc nhất $ax + b = 0$ với a, b nhập từ bàn phím

Hướng dẫn:

- Khai báo hai biến a, b kiểu số thực để lưu hệ số của phương trình do người dùng nhập tùy ý từ bàn phím. Nhập a, b
- Giải thuật: Xét đủ 2 trường hợp $a = 0$ và $a \neq 0$
- Hãy vận dụng lệnh if và if.. else để cài đặt giải thuật trên sao cho chương trình rõ ràng nhất



```

#include <stdio.h>
#include <conio.h>
int main()
{
    float a,b;
    //SV tu code nhap gia tri a;
    //SV tu code nhap gia tri b;
    if ( a ==0 )
        if ( b==0 )
            printf ( "  ");
        else
            printf ( " ");
    else
        printf ("Phuong trinh co nghiem x= %f \n", -b/a);
    return 0;
}
  
```

Câu 3: Viết chương trình giải phương trình bậc hai $ax^2 + bx + c = 0$. Với a, b, c nhập từ bàn phím

Hướng dẫn:

- Khai báo 3 biến a, b, c kiểu số thực. Nhập các hệ số a, b, c

- Xét $a = 0$. Phương trình trở thành bậc nhất, giải và biện luận theo b, c (tương tự bài 2)
- Xét $a \neq 0$. Tính delta, $d = b^2 - 4ac$. Xét các trường hợp $d = 0$, $d < 0$ và $d > 0$
- Nếu $d < 0$ thì phương trình vô nghiệm
- Nếu $d = 0$ thì phương trình có nghiệm kép $x_1 = x_2 = -b/(2a)$
- Nếu $d > 0$ thì phương trình có 2 nghiệm phân biệt
- Hàm tính căn bậc hai $\text{sqrt}(\text{số})$ nằm trong thư viện `<math.h>`.

Câu 4: Nhập vào 3 số nguyên dương a, b, c . Kiểm tra xem 3 số đó có lập thành tam giác không? Nếu có hãy tính chu vi và diện tích của tam giác theo công thức:

- Chu vi $CV = a + b + c$
- Diện tích $S = \text{sqrt}(p(p-a)(p-b)(p-c))$, trong đó: $p = CV/2$
- Xuất các kết quả ra màn hình

Hướng dẫn:

- a, b, c là số nguyên dương $\Rightarrow$ khai báo a, b, c kiểu *unsigned int*
- Điều kiện để 3 số lập thành tam giác: tổng 2 cạnh phải lớn hơn cạnh còn lại
Vậy: a, b, c lập thành 3 cạnh của tam giác khi và chỉ khi $a + b > c$ và $a + c > b$ và $b + c > a$
- Tính diện tích và chu vi theo công thức đã cho
- Hàm căn bậc hai $\text{sqrt}(x)$ trong thư viện `<math.h>`.

Câu 5: Nhập vào 3 số nguyên dương a, b, c . Kiểm tra xem 3 số đó có lập thành tam giác không? Nếu có hãy cho biết tam giác đó thuộc loại nào? (Cân, vuông, đều, ...).

Hướng dẫn:

- a, b, c là số nguyên dương $\Rightarrow$ khai báo a, b, c kiểu *unsigned int*. Chuỗi định dạng của kiểu nguyên dương là `%u`
- Điều kiện để 3 số lập thành tam giác: tổng 2 cạnh phải lớn hơn cạnh còn lại. Vậy: a, b, c lập thành 3 cạnh của tam giác $\Leftrightarrow a + b > c$ và $a + c > b$ và $b + c > a$
- Xét loại tam giác:

- Tam giác cân $\Leftrightarrow a = b$ hoặc $b = c$ hoặc $c = a$
- Tam giác đều $\Leftrightarrow a=b=c \Leftrightarrow (a == b \ \&\& \ b == c)$
- Tam giác vuông $\Leftrightarrow a^2 = b^2 + c^2$ hoặc $b^2 = a^2 + c^2$ hoặc $c^2 = a^2 + b^2$.

Câu 6: Viết chương trình nhập số nguyên có hai chữ số, hiển thị cách đọc số đó

Hướng dẫn:

- Khai báo và nhập số nguyên n
- Nếu $9 < n < 100$ thì thực hiện đọc số n
 - Lấy chữ số hàng chục $n/10$. Sử dụng câu lệnh *switch* để đọc chữ số hàng chục
 - Lấy chữ số hàng đơn vị $n\%10$. Sử dụng câu lệnh *switch* để đọc chữ số hàng đơn vị
- Ngược lại thì xuất ra thông báo "số nhập vào không phải là số có hai chữ số".

Câu 7: Viết chương trình nhập vào tháng của một năm, cho biết số ngày của tháng đó. Nếu tháng nhập vào <1 hoặc >12 thì thông báo "*Không tồn tại tháng này*"

Hướng dẫn:

- Khai báo biến và nhập vào tháng
- Các tháng 1, 3, 5, 7, 8, 10, 12 là tháng có 31 ngày
- Các tháng 4, 6, 9, 11 là tháng có 30 ngày
- Nếu là tháng 2 thì yêu cầu nhập thêm năm, nếu là năm nhuận thì tháng 2 có 29 ngày, còn lại là 28 ngày
- Năm nhuận là năm chia hết cho 4 (hoặc những năm có 2 số cuối là số 0 thì các bạn lấy số năm chia cho 400, nếu chia hết thì năm đó là năm có nhuận)
- Nếu tháng nhập vào không thuộc các tháng trên thì thông báo "không tồn tại tháng này".

2.3 THỰC HÀNH NÂNG CAO

Câu 8: Viết chương trình tính tiền điện sinh hoạt (đã bao gồm thuế VAT 10%) trong tháng của 1 hộ gia đình khi sử dụng N (kWh) tiêu thụ với ($0 \leq N \leq 104$) và được làm tròn đến số nguyên gần nhất. Biết rằng:

	ĐƠN GIÁ (đồng/kWh)	SẢN LƯỢNG (kWh)
Bậc thang 1	1678	50
Bậc thang 2	1734	50
Bậc thang 3	2014	100
Bậc thang 4	2536	100
Bậc thang 5	2834	100
Bậc thang 6	2927	0

Hướng dẫn:

Nhập số nguyên $N = 283$

Phân tích N nằm ở các bậc thang:

- 50 số điện (bậc 1)
- 50 số tiếp theo (bậc 2)
- 100 số (bậc 3)
- 83 số còn lại (bậc 4)

Tính giá tiền chưa VAT theo từng bậc * sản lượng tương ứng

Tính giá tiền có VAT (10%)

=> Kết quả tiền điện: 582488.

2.4 BÀI TẬP VỀ NHÀ

Câu 9: Viết chương trình nhập số nguyên có ba chữ số, hiển thị cách đọc số đó

Câu 10: Viết chương trình nhập một số nguyên dương có 5 chữ số, hãy kiểm tra số vừa nhập có tăng dần đều từ trái sang phải hay không.

Câu 11: Viết chương trình nhập vào tháng của một năm, cho biết tháng đó thuộc quý mấy của năm.

Câu 12: Viết chương trình nhập vào ba số thực x, y, z , hãy kiểm tra số thực đã nhập có phải số âm hay không? Nếu là số âm thì lấy giá trị tuyệt đối của số thực.

Câu 13: Cho hệ phương trình

$$ax + by = c$$

$$dx + ey = f$$

Viết chương trình giải hệ phương trình. Biết rằng: các hệ số a, b, c, d, e, f là các số nguyên được nhập từ bàn phím.

BÀI 3: CẤU TRÚC LẶP

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Nắm vững cú pháp, cách thức hoạt động của cấu trúc lặp *for*.
- Nắm vững cú pháp, cách thức hoạt động của cấu trúc lặp *while*.
- Nắm vững cú pháp, cách thức hoạt động của cấu trúc lặp *do ... while*.

3.1 TÓM TẮT LÝ THUYẾT

3.1.1 Cấu trúc *for*

Cú pháp:

```
for (biểu thức khởi gán; biểu thức điều kiện; biểu thức tăng/giảm)
{
    khối lệnh thực hiện công việc;
}
```

Ý nghĩa:

Bước 1: Khởi gán cho biểu thức 1

Bước 2: Kiểm tra điều kiện của biểu thức 2.

- Nếu biểu thức 2 $\neq 0$ thì cho thực hiện các lệnh của vòng lặp, thực hiện biểu thức 3. Quay trở lại bước 2.
- Ngược lại thoát khỏi lặp.

3.1.2 Cấu trúc *while*

Cú pháp:

```
<Khởi gán>
while (biểu thức điều kiện)
{
```

```
Lệnh/ khối lệnh;  
Tăng/giảm chỉ số lặp;  
}
```

Ý nghĩa: Cách hoạt động của while giống for.

3.1.3 Cấu trúc do ... while

Cú pháp:

```
do  
{  
    Khối lệnh thực hiện công việc;  
} while (biểu thức điều kiện) ;
```

Ý nghĩa: Thực hiện khối lệnh cho đến khi biểu thức điều kiện có giá trị sai (có giá trị bằng 0) thì dừng.

❖ **Lưu ý:**

Lặp while kiểm tra điều kiện trước khi thực hiện lặp, còn vòng lặp do...while thực hiện lệnh lặp rồi mới kiểm tra điều kiện. Do đó vòng lặp do...while thực hiện lệnh ít nhất một lần.

3.2 THỰC HÀNH CƠ BẢN

Câu 1: Viết chương trình thực hiện:

- Xuất ra màn hình 10 dòng: "XIN CHAO CAC BAN"
- Xuất ra màn hình n dòng: ""XIN CHAO CAC BAN", với n nhập từ bàn phím.

Hướng dẫn:

- Xác định số lần lặp: 10 lần. Xác định công việc lặp: xuất câu thông báo "XIN CHAO CAC BAN" ra màn hình. Đoạn code sau minh họa cách dùng các vòng lặp for, while, do ... while.

```
//Cách 1: Sử dụng vòng lặp for  
int i;  
for(i=1; i<=10; i++)  
    printf("XIN CHAO CAC BAN\n");
```

```
//Cách 2: Sử dụng vòng lặp while -----
i=1;
while (i<=10)
{
    printf("XIN CHAO CAC BAN\n");
    i=i+1; //hoặc i++; hoặc i+=1;
}
//Cách 3: Sử dụng vòng lặp do ... while -----
i=1;
do{
    printf("XIN CHAO CAC BAN\n");
    i=i+1;
}while (i<=10);
```

Câu 2: Viết chương trình nhập vào một số nguyên $n > 0$, hãy:

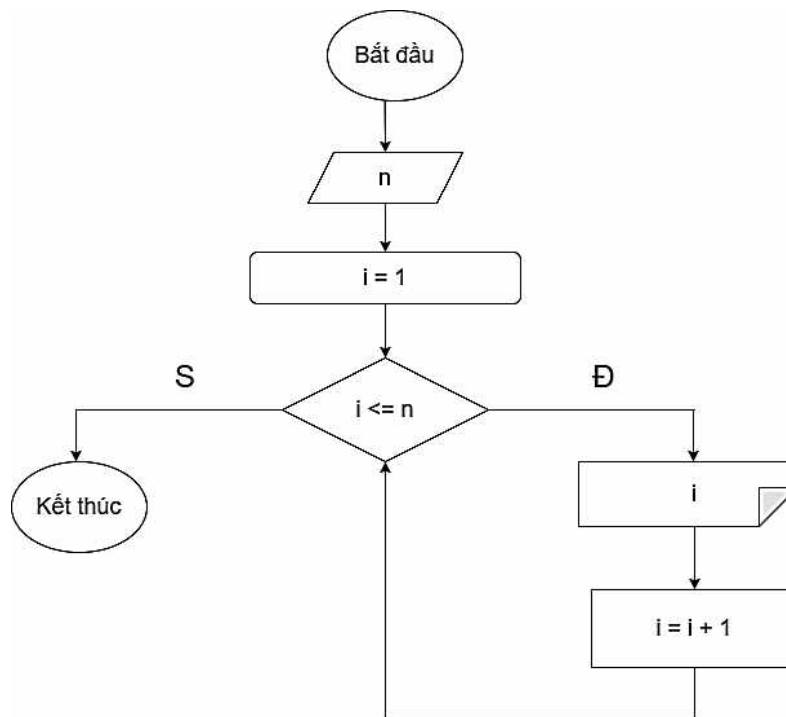
- Xuất ra màn hình các số trong phạm vi từ 1 đến n
- Xuất ra màn hình các số chẵn trong phạm vi từ 1 đến n
- Xuất ra màn hình các số lẻ không chia hết cho 3 trong phạm vi từ 1 đến n
- Tính các biểu thức sau:
 - $S1 = 1 + 2 + \dots + n$
 - $S2 = -1 + 2 - 3 + 4 - \dots + (-1)^n n$.
 - $S3 = 1/2 + 2/3 + 3/4 \dots + n/(n+1)$
 - $S4 = x^n$ (x là số thực nhập từ bàn phím).
- Tính tổng các chữ số của n . Ví dụ: $n = 125$, tổng các chữ số là 8.

Hướng dẫn:

Nhập số nguyên $n > 0$, nếu nhập sai thì bắt nhập lại.

```
do {
    printf (" Nhập vào số phần tử " );
    scanf( "%d" , &n);
    if(n<=0)
        printf( " nhap sai, nhap lai " );
}while ( n<=0 );
```

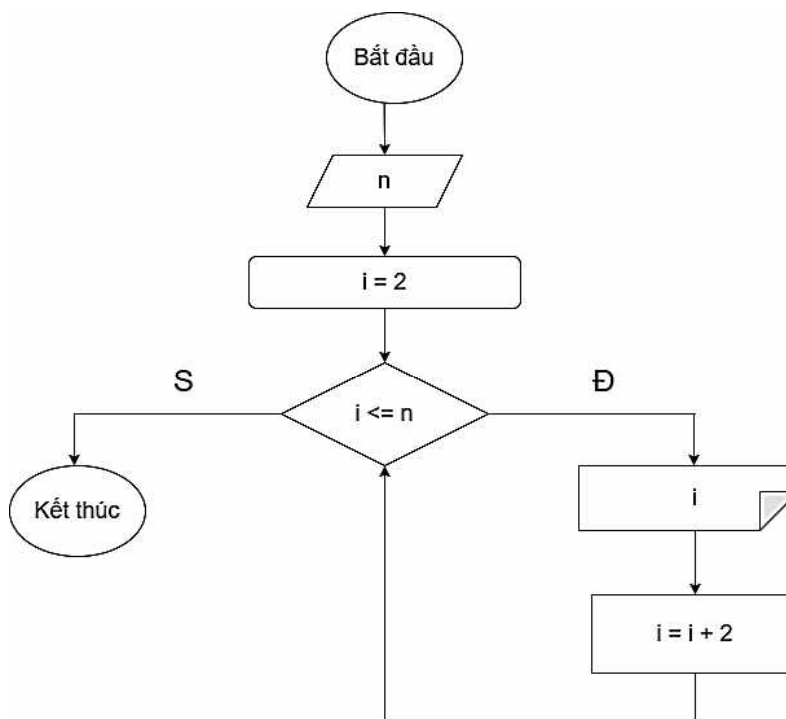
a. Xuất ra màn hình các số trong phạm vi từ 1 đến n.



- Câu a, b, c, d có thể áp dụng cấu trúc lặp for/while.

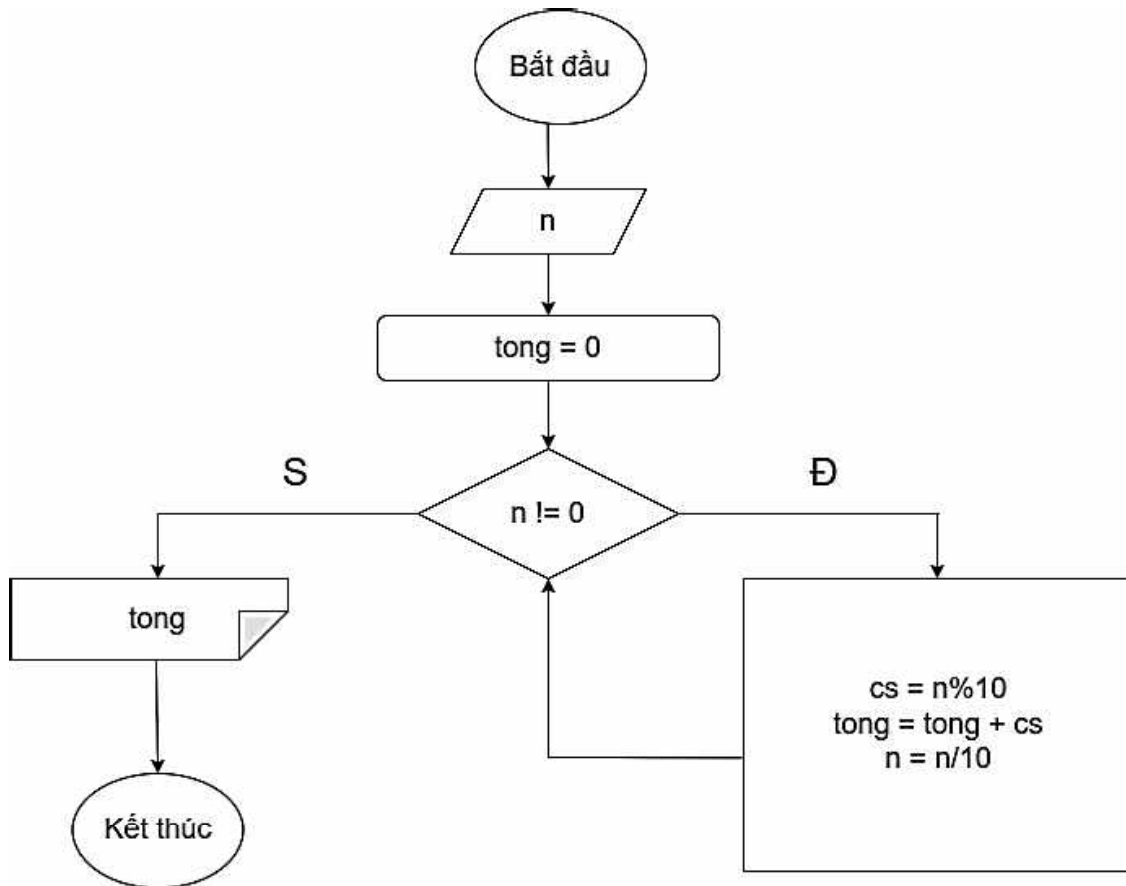
Hướng dẫn câu b

b. Xuất ra màn hình các số chẵn trong phạm vi từ 1 đến n.



e. Tính tổng các chữ số của n

- ❖ Lần lượt lấy từng chữ số của số n và cộng dồn vào biến tổng.
- Giả sử, khai báo biến lưu tổng các chữ số là **tong**, biến lưu từng chữ số của n là **cs**.
- Thuật toán như sau:



Câu 3: Viết chương trình thực hiện:

- a. Nhập một số nguyên n sao cho $0 < n \leq 100$. Nếu nhập sai thì yêu cầu nhập lại
- b. Đếm số ước của n . Nếu đếm bằng 2 thì xuất ra màn hình " n là số nguyên tố", ngược lại xuất ra " n không phải là số nguyên tố"
- c. Tính tổng các ước của n (không tính chính nó). Nếu tổng các ước đúng bằng n thì xuất ra màn hình " n là số hoàn thiện", ngược lại xuất ra " n không là số hoàn thiện".

Hướng dẫn:

- **Điều kiện nhập lại n : $n \leq 0$ hoặc $n > 100$.**


```
do {

    printf ( " Nhập vào số phần tử " );

    scanf( " %d" , &n);

    if ( ... )

        printf ( " nhap sai, nhap lai " );

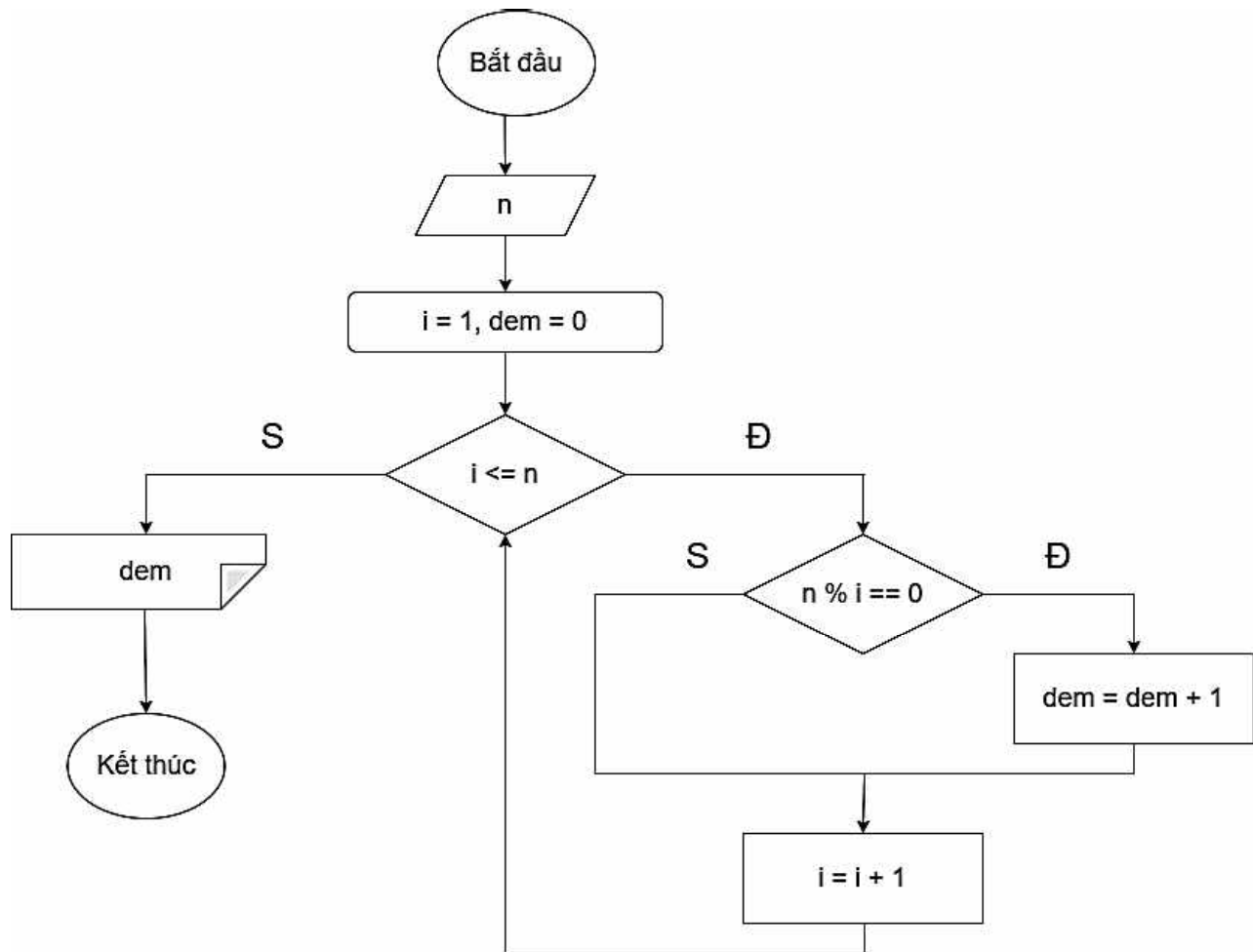
}while ( n<=0 || n >= 100 );
```

- Sử dụng phép chia lấy dư (%) để xác định số nào là ước của n .

a. Đếm số ước của n

Khai báo biến đếm là d , đầu tiên gán $d=0$;

Lần lượt lấy n chia lấy phần dư (%) cho các số từ 1 đến n , nếu số nào có số dư là 0 thì ta tăng biến đếm lên 1 đơn vị -> dùng vòng lặp



Câu 4: Viết chương trình thực hiện:

- Nhập $n > 0$
- Liệt kê các số nguyên tố trong phạm vi từ 1 đến n
- Đếm số lượng số nguyên tố trong phạm vi từ 1 đến n
- Tính tổng các số nguyên tố trong phạm vi từ 1 đến n .

Hướng dẫn

b. Liệt kê các số nguyên tố trong phạm vi từ 1 đến n .

```
for ( int i=2 ; i<=n ; i++ )
    kiểm tra nto ( i ) nếu đúng thì in ra số i ;
```

c. Đếm số lượng các số nguyên tố trong phạm vi từ 1 đến n .

```
int d=0;
for ( int i=2 ; i<=n ; i++ )
    kiểm tra nto ( i ) nếu là số nguyên tố, tăng biến đếm lên 1 đơn vị ;
```

3.3 THỰC HÀNH NÂNG CAO

Câu 5: Để sản xuất một màn hình TV có kích thước n pixel, nhà sản xuất yêu cầu tính kích thước của màn hình TV sao cho chiều dài và chiều rộng ít chênh lệch nhất.

Input	Output
8	2 4
64	8 8

Câu 6: Viết chương trình hiển thị ra màn hình n dòng ($0 < n \leq 10$):

a.	b.	c.	d.
*****	*	*****	*
*****	**	****	***
*****	***	***	*****
*****	****	**	*****
*****	*****	*	*****

Gợi ý:

- a. Hiển thị trên n dòng, mỗi dòng có k dấu *, k tùy ý người dùng nhập
- b. Hiển thị trên n dòng, dòng thứ i có i dấu *

Hướng dẫn:

- Câu a: Sử dụng 2 vòng for lồng nhau, vòng for thứ nhất để lặp xét từng dòng (lặp n lần), vòng for thứ hai để lặp xuất ra dấu * theo số lượng yêu cầu (lặp k lần)
- Câu b: Sử dụng 2 vòng for lồng nhau, vòng for thứ nhất để lặp xét từng dòng (lặp n lần), vòng for thứ hai để lặp xuất ra dấu * theo số lượng yêu cầu (lặp i lần).

3.4 BÀI TẬP VỀ NHÀ

Câu 7: Viết chương trình đoán số đã cho có hai chữ số ($0 < x < 100$). Biết rằng:

- Giá trị x được random ($0 < x < 100$)

Gợi ý: sử dụng thư viện `#include <time.h>` và `#include <stdlib.h>`

```
srand((int)time(0))
```

```
minN + rand() % (maxN + 1 - minN)
```

- Nhập giá trị dự đoán x ($0 < n < 100$)
- Kiểm tra nếu $n < x$ thì xuất thông báo "thấp hơn", nếu $n > x$ thì xuất thông báo "cao hơn"
- Sử dụng vòng lặp để biết rằng người dùng muốn nhập tiếp hay không (1: tiếp tục; 0: dừng).

Câu 8: Cho hai số nguyên dương x và y . Viết chương trình tìm ước số chung lớn nhất của x và y .

Câu 9: Cho hai số nguyên dương x và y . Viết chương trình tìm bội số chung nhỏ nhất của x và y .

Câu 10: Viết chương trình hiển thị bảng cửu chương ra màn hình. Lưu ý: hiển thị theo hàng ngang từ cửu chương 2 tới cửu chương 9.

BÀI 4: HÀM

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

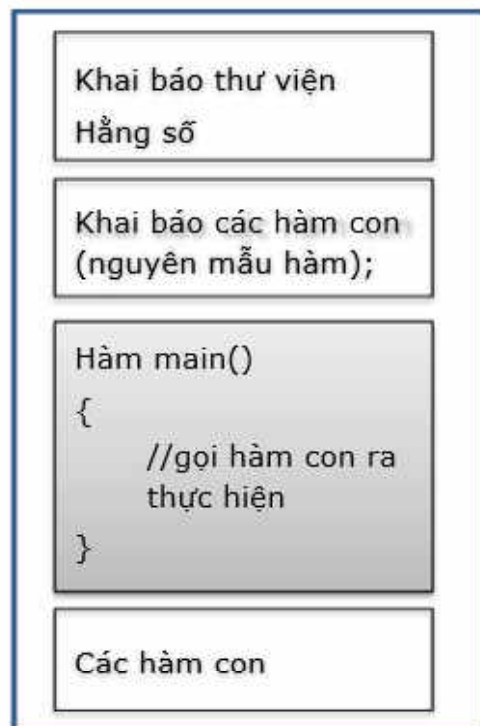
- *Khái niệm về hàm (function) trong C.*
- *Cách xây dựng và cách sử dụng hàm trong C.*

4.1 TÓM TẮT LÝ THUYẾT

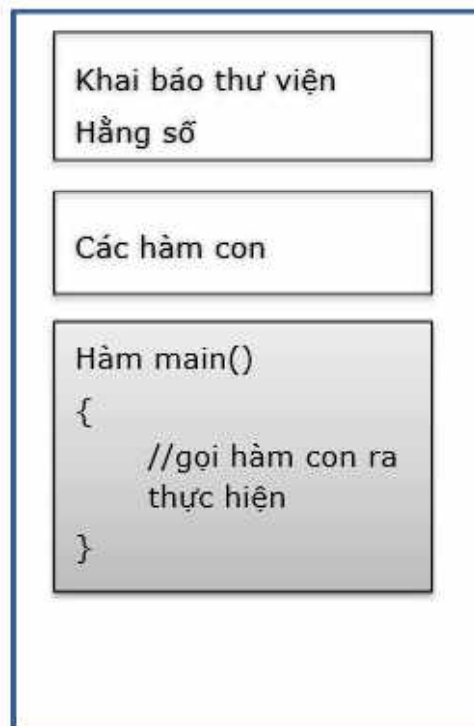
Hàm là một đoạn chương trình độc lập thực hiện trọn vẹn một công việc nhất định sau đó trả về giá trị cho chương trình gọi nó, hay nói cách khác hàm là sự chia nhỏ của chương trình.

4.1.1 Cấu trúc một chương trình C theo hàm

Cách 1:



Cách 2:



- Đầu chương trình là các khai báo về sử dụng thư viện, khai báo hằng số, khai báo các biến toàn cục và khai báo các kiểu dữ liệu tự định nghĩa, khai báo hàm con (các nguyên mẫu hàm).
- Nguyên mẫu hàm:

`<Kiểu dữ liệu của hàm> Tên hàm ([danh sách các tham số]);`

- Nguyên mẫu hàm thực chất là dòng đầu của hàm thêm dấu chấm phẩy (;) vào cuối, tuy nhiên tham số trong nguyên mẫu hàm có thể bỏ phần tên.
- Hàm chính (`main()`): Chứa các biến, các lệnh và các lời gọi hàm cần thiết trong chương trình.
- Các hàm con được sử dụng nhằm mục đích:
 - Khi có một công việc giống nhau cần thực hiện ở nhiều vị trí.
 - Khi cần chia một chương trình lớn phức tạp thành các đơn thể nhỏ (hàm con) để chương trình được trong sáng, dễ hiểu trong việc xử lý, quản lý việc tính toán và giải quyết vấn đề.

4.1.2 Cách xây dựng một hàm con

- Định nghĩa một hàm bao gồm:
 - Khai báo kiểu hàm
 - Đặt tên hàm
 - Khai báo các tham số
 - Các câu lệnh cần thiết để thực hiện chức năng của hàm.
- Có 2 loại hàm:
 - **void**: Hàm không trả về trị. Những hàm loại này thường rơi vào những nhóm chức năng: **Nhập / xuất dữ liệu, thông kê, sắp xếp, liệt kê.**
 - **Kiểu dữ liệu cơ bản (rời rạc/ liên tục) hay kiểu dữ liệu có cấu trúc**: Hàm trả về trị. Sau khi thực hiện xong, kết quả được lưu lại. Những hàm loại này thường được sử dụng trong các trường hợp: **Đếm, kiểm tra, tìm kiếm, tính trung bình, tổng, tích, ...**

Hàm không trả về trị:

```
void Ten_Ham(ds tham so neu co)
{
    Khai báo các biến cục bộ;
    Các câu lệnh / khối lệnh hay
    lời gọi đến hàm khác;
}
```

Hàm trả về trị:

```
Kieu_dl Ten_Ham(ds tham so neu co)
{
    Kiểu_dl kq;
    Khai báo các biến cục bộ;
    Các câu lệnh / khối lệnh hay
    lời gọi đến hàm khác;
    return kq;
}
//kiểu dữ liệu của hàm là kiểu của
kết quả trả về.
```

4.1.3 Tham số của hàm

Xác định dựa vào dữ liệu đầu vào của bài toán (Input). Gồm 2 loại :

- Tham trị: Giá trị của biến tham số đầu vào không thay đổi sau khi hàm thực hiện xong. Tham số dạng này chỉ mang ý nghĩa là dữ liệu đầu vào.

Ví dụ: `int KiemTraNguyenTo(int n);`

- Tham biến: Có sự thay đổi giá trị của tham số trong quá trình thực hiện và cần lấy lại giá trị đó sau khi ra khỏi hàm. Ứng dụng của tham số loại này có thể là dữ liệu đầu ra (kết quả) hoặc cũng có thể vừa là dữ liệu đầu vào vừa là dữ liệu đầu ra.

Ví dụ:

Hàm nhập số nguyên: `void NhapSoNguyen(int &n);`

Hàm hoán vị giá trị hai số: `void HoanVi(int &a, int &b);`

4.1.4 Cách gọi hàm con ra thực hiện

Một hàm chỉ được thực thi khi ta có một lời gọi đến hàm đó.

Cú pháp gọi hàm:

```
<Tên hàm>([Danh sách các tham số])
```

4.2 THỰC HÀNH CƠ BẢN

Câu 1: Viết chương trình thực hiện các chức năng sau (dùng hàm)

- Nhập vào một số nguyên n ($0 < n < 100$)
- Kiểm tra n có phải là số nguyên tố không?
- Liệt kê các số nguyên tố trong phạm vi từ 1 đến n
- Đếm số lượng số nguyên tố trong phạm vi từ 1 đến n
- Tính tổng các số nguyên tố trong phạm vi từ 1 đến n
- Tính trung bình cộng các số nguyên tố trong phạm vi từ 1 đến n .

Hướng dẫn:

- Nhập vào một số nguyên n ($0 < n < 100$).

```
void NhapSoNguyen(int &n)
{
    Lặp công việc{
        Nhập n
        Nếu  $n \leq 0$  hoặc  $n \geq 100$  thì thông báo nhập sai, hãy nhập lại.
    } Chừng nào ( $n \leq 0$  hoặc  $n \geq 100$ );
}
```

- Kiểm tra n có phải là số nguyên tố không?

Hàm trả về 1: nếu n là số nguyên tố

Hàm trả về 0: nếu n không phải là số nguyên tố

int KTNT(int n) {...}

- Liệt kê các số nguyên tố trong phạm vi từ 1 đến n

void LietKeNT(int n) {...}

- Đếm số lượng số nguyên tố trong phạm vi từ 1 đến n

Hàm trả về số lượng số nguyên tố đã đếm được trong phạm vi từ 1 đến n

int DemNT(int n) {...}

- e. Tính tổng các số nguyên tố trong phạm vi từ 1 đến n

```
int TongNT(int n) {...}
```

- f. Tính trung bình cộng các số nguyên tố trong phạm vi từ 1 đến n .

```
float tbcNT(int n) {...}
```

❖ **Lưu ý:**

- Với mỗi hàm, ta test thử trong hàm *main()*, sau khi chạy ra kết quả đúng ta mới làm tiếp những hàm khác.
- Cách gọi thực hiện hàm con trong *main()*: chỉ gọi **tên_hàm (các tham số thực tương ứng)**

Câu 2: Viết chương trình sử dụng cách viết hàm để thực hiện chọn lựa công việc:

- Giải phương trình bậc 1: $ax+b=0$
- Kiểm tra một số nguyên có là số hoàn thiện không?
- Liệt kê các số hoàn thiện trong phạm vi từ 1.. n (n do người dùng nhập)
- Tìm ước chung lớn nhất của hai số nguyên a, b nhập từ bàn phím
- Thoát khỏi chương trình.

Hướng dẫn:

- Hãy xác định bạn cần viết bao nhiêu hàm, đó là những hàm nào?
- Trong hàm *main*, gọi hàm như bình thường. Sau khi chương trình đã chạy được và không còn lỗi, hãy sửa đổi để có chương trình thực hiện theo chức năng, tham khảo đoạn code sau:

```
//Xuất ra menu công việc
void XuatMenu()
{
    printf("1: Giai phuong trinh bac 1\n");
    printf("2: Kiem tra so hoan thien\n");
    printf("3: Liet ke so hoan thien trong pham vi tu 1 den n\n");
    printf("4: Tim uoc chung lon nhat cua hai so nguyen\n");
    printf("0: Thoat\n");
}
```



```
}  
//-----  
//đoạn code này viết trong hàm main()  
//dùng một biến nguyên để lưu công việc mà người dùng chọn  
int chon;  
do{  
    XuatMenu(); //hiển thị menu công việc cho người dùng chọn  
    printf("Hay chon cong viec:"); scanf("%d", &chon);  
    switch (chon) {  
        case 1:  
            float a, b;  
            //nhập hệ số a, b  
            //Gọi hàm giải pt bậc 1  
            break;  
        case 2:  
            //khai báo và nhập số nguyên  
            //gọi hàm Kiểm tra x có phải số hoàn thiện không  
            break;  
        case 3:  
            //khai báo và nhập số nguyên n  
            //gọi hàm xuất các số hoàn thiện trong phạm vi 1..n  
            break;  
        case 4:  
            //khai báo và nhập số 2 số nguyên a, b  
            //gọi hàm tìm ước chung lớn nhất  
            break;  
        default: chon=0;  
    }  
} while (chon!=0);
```

Câu 3: Viết chương trình nhập từ bàn phím 2 số a, b và một ký tự ch

Nếu:

ch là "+" thì thực hiện phép tính $a + b$ và in kết quả lên màn hình

ch là "-" thì thực hiện phép tính $a - b$ và in kết quả lên màn hình

ch là "*" thì thực hiện phép tính $a * b$ và in kết quả lên màn hình

ch là "/" thì thực hiện phép tính a / b và in kết quả lên màn hình.

4.3 THỰC HÀNH NÂNG CAO

Câu 4: Viết hàm thực hiện:

- Viết hàm tìm số Fibonacci thứ n
- Viết hàm liệt kê các số Fibonacci nhỏ hơn n (với n là số nguyên tố).

Gợi ý:

$$F_0 = 0; F_1 = 1; F_2 = 1; F_3 = 2; F_n = F_{(n-1)} + F_{(n-2)} \text{ với } n > 2$$

Dãy Fibonacci: 0 1 1 2 3 5 8 13 21...

Câu 5: Viết chương trình tính tiền lương ngày công cho công nhân, cho biết trước giờ vào ca, giờ ra ca của mỗi người.

Giả sử rằng:

- Tiền trả cho mỗi giờ trước 12 giờ là 6000đ và sau 12 giờ là 7500đ.
- Giờ vào ca sớm nhất là 6 giờ sáng và giờ ra ca trễ nhất là 18 giờ (*Giả sử giờ nhập vào nguyên*).

4.4 BÀI TẬP VỀ NHÀ

Câu 6: Viết hàm kiểm tra số nguyên n vừa nhập có phải số chính phương hay không? Biết rằng: số chính phương là số có **căn của n là một số nguyên**.

Câu 7: Viết hàm liệt kê các số chính phương trong phạm vi từ 1 tới n . Biết rằng: n được nhập từ bàn phím.

Câu 8: Viết hàm xuất ra "họ tên học sinh - điểm trung bình - kết quả xếp loại" của học sinh. Biết rằng:

- Nhập họ tên học sinh. Gợi ý: sử dụng hàm **gets(hoten)**
- Nhập ba điểm của học sinh; toán – văn – anh văn (toán – văn – anh văn là ba số thực được nhập từ bàn phím).

Câu 9: Viết hàm tính tổng các chữ số chẵn của số nguyên dương n .

Câu 10: Viết hàm kiểm tra các chữ số của số nguyên dương n có giảm dần đều hay không?

BÀI 5: MẢNG MỘT CHIỀU

Sau khi thực hành xong bài này, sinh viên có thể nắm được:

- Cách khai báo dữ liệu kiểu mảng
- Các thao tác nhập xuất, các kỹ thuật thao tác trên mảng.
- Luyện tập cài đặt chương trình con (hàm).

5.1 TÓM TẮT LÝ THUYẾT

Mảng một chiều thực chất là một biến được cấp phát bộ nhớ liên tục và bao gồm nhiều biến thành phần.

Các thành phần của mảng là tập hợp các biến có cùng kiểu dữ liệu và cùng tên. Do đó để truy xuất các biến thành phần, ta dùng cơ chế chỉ mục.

Khai báo:

```
Kiểu_dữ_liệu Tên_mảng[Kích_thước];
```

Ví dụ:

```
int a[100]; // Khai báo mảng số nguyên a gồm 100 phần tử
```

```
float b[50]; // Khai báo mảng số thực b gồm 50 phần tử
```

Sử dụng:

```
Tên_biến_mảng[chỉ_số];
```

Ví dụ : `int A[5]` // Khai báo mảng A gồm tối đa 5 phần tử nguyên.

Chỉ số (số thứ tự)

0	1	2	3	4
A[0]	A[1]	A[2]	A[3]	A[4]

5.2 THỰC HÀNH CƠ BẢN

Câu 1: Viết chương trình thực hiện:

- Nhập mảng số nguyên gồm n phần tử ($0 < n \leq 10$)
- Xuất mảng vừa nhập.

Hướng dẫn:

Chương trình minh hoạ

```
#include <conio.h>
#include <stdio.h>
#define MAX 100
/* =====
 * Hàm nhập số lượng phần tử cho mảng */
void NhapSL(int &n)
{
    do{
        printf ("Nhap so phan tu 0<sl<100: ");
        scanf ("%d ", &n);
    } while (n<=0 || n>100);
}
/* =====
 * Hàm nhập giá trị cho từng phần tử trong mảng */
void NhapMang (int a[], int n)
{
    for (int i = 0; i < n; i ++){
        printf (" a[%d] = ", i);
        scanf ("%d ", &a[i]);
    }
}
/* =====
 * Hàm xuất các phần tử của mảng ra màn hình */
void XuatMang (int a[], int n)
{
    printf ("\nMang gom cac phan tu:\n ");
    for (int i = 0; i < n; i ++){
        printf (" %5d", a[i]);
    }
}
/* =====
 * Hàm Main
 * ===== */
```

```
int main()
{
    int a[MAX], n;
    NhapMang (a,n);
    XuatMang (a,n);
    return 0;
}
```

Câu 2: Làm tiếp theo trong chương trình của câu 1 với các yêu cầu sau:

- a. Xuất các phần tử chia hết cho 3 có trong mảng
- b. Đếm số lượng số dương có trong mảng
- c. Tính tổng các số trong mảng
- d. Tính trung bình cộng của mảng
- e. Tính trung bình cộng các phần tử dương có trong mảng
- f. Xuất các số nguyên tố có trong mảng
- g. Đếm số lượng số nguyên tố có trong mảng
- h. Tính tổng các số nguyên tố có trong mảng
- i. Tính trung bình cộng các số nguyên tố có trong mảng
- j. Tìm phần tử dương đầu tiên. Nếu mảng không có số dương nào thì xuất thông báo "mảng không chứa số dương"
- k. Tìm phần tử âm cuối cùng. Nếu mảng không có số âm nào thì xuất thông báo "mảng không chứa số âm"
- l. Tìm giá trị phần tử lớn nhất (nhỏ nhất) trong mảng
- m. Kiểm tra mảng đã nhập có đối xứng hay không.

Hướng dẫn:

- a. Xuất các phần tử chia hết cho 3 có trong mảng.**

Giải thuật:

- Đi từ đầu mảng đến cuối mảng
 - `for ( int i=0; i<n ; i++)`

- Kiểm tra từng phần tử của mảng xem có chia hết cho 3 không

- `if (a[i] %3==0 )`

Nếu có thì in phần tử đó ra

```
printf ( " %d", a[i] );
```

c. Tính tổng các số trong mảng.

Giải thuật:

- Khai báo một biến `s` để lưu trữ tổng `s = 0`;
- Đi từ đầu mảng đến cuối mảng
 - `for ( int i=0; i<n ; i++)`
- cộng dồn các phần tử `a[i]` và lưu vào biến `s`
 - `s= s+ a[i] ;`

j. Tìm phần tử dương đầu tiên

Giải thuật :

- Đi từ đầu mảng đến cuối mảng
 - `for ( int i=0; i<n ; i++)`
- Kiểm tra từng phần tử của mảng xem có dương không
 - `if (a[i] > 0 )`

Nếu có thì xuất phần tử đó ra và dừng chương trình

```
return a[i] ;
```

5.3 THỰC HÀNH NÂNG CAO

Câu 3: Viết chương trình thực hiện:

- Nhập mảng số nguyên gồm n phần tử ($3 \leq n \leq 20$)
- Thêm phần tử **D** ở đầu mảng (**D** nhập từ bàn phím)
- Thêm phần tử **C** ở cuối mảng (**C** nhập từ bàn phím)

- d. Thêm phần tử **K** ở vị trí **T** trong mảng (**K** nhập từ bàn phím; **T** phải thoả điều kiện $0 \leq t < n$)
- e. Xoá 1 phần tử đầu tiên trong mảng
- f. Xoá 1 phần tử cuối cùng trong mảng
- g. Xoá phần tử ở vị trí **T** trong mảng (**T** phải thoả điều kiện $0 \leq t < n$).

Lưu ý: Khi thêm hoặc xoá bất kì phần tử trong mảng thì số lượng phần tử sẽ thay đổi.

Hướng dẫn:

*//Hàm thêm phần tử **D** ở đầu mảng*

```
void ThemDau(int a[],int &n)
{
    //sinh viên tự code nhập phần tử k cần thêm
    n++;
    for(int i=n-1; i>0; i--)
    {
        a[i] = a[i-1];
    }
    a[0]=k;
}
```

*//Hàm thêm phần tử **D** ở đầu mảng*

```
void ThemPTK(int a[],int &n)
{
    int k, t;//k la vi tri va t la gia tri a[k]
    //sinh viên tự code nhập t và ràng buộc (0<k<n)
    //sinh viên tự code nhập giá trị t
    n++;
    for(int i=n-1; i>k; i--)
    {
        a[i] = a[i-1];
    }
}
```



```
a[k]=t;  
}
```

// Xoá 1 phần tử đầu tiên trong mảng

```
void XoaPhanTu (int a[],int &n){  
    for (int i=0; i<n-1;i++){  
        a[i]=a[i+1];  
    }  
    n--;  
}
```

Câu 4: Viết chương trình thực hiện:

- h. Nhập vào mảng a gồm n phần tử, trong quá trình nhập kiểm tra các phần tử nhập vào không được trùng, nếu trùng thông báo và yêu cầu nhập lại
- i. Xuất các phần tử trong mảng
- j. Xuất ra màn hình các phần tử là số chính phương nằm tại những vị trí lẻ trong mảng
- k. Xuất ra vị trí của các phần tử có giá trị lớn nhất
- l. Viết hàm tính tổng các phần tử nằm ở vị trí chẵn trong mảng
- m. Viết hàm sắp xếp mảng theo thứ tự tăng dần.

Yêu cầu chương trình thực hiện theo menu chức năng.

5.4 BÀI TẬP VỀ NHÀ

Câu 5: Viết chương trình thực hiện:

- a. Nhập mảng số thực gồm n phần tử ($0 < n \leq 100$)
- b. Xuất mảng vừa nhập.
- c. Tính tổng trung bình cộng các phần tử âm có trong mảng. Nếu không có thì xuất thông báo "*Không có phần tử âm nào trong mảng*".
- d. Tìm vị trí phần tử âm đầu tiên trong mảng, nếu không có phần tử âm nào thì xuất thông báo "*Không có phần tử âm nào trong mảng*".

- e. Kiểm tra mảng nhập vào có đối xứng hay không.
- f. Xuất các vị trí của phần tử lớn nhất trong mảng.

BÀI 6: MẢNG HAI CHIỀU

Sau khi học xong bài này, sinh viên có thể nắm được:

- Cách khai báo dữ liệu kiểu mảng 2 chiều;
- Các thao tác nhập xuất, các kỹ thuật thao tác trên mảng 2 chiều.

6.1 TÓM TẮT LÝ THUYẾT

Mảng hai chiều thực chất là mảng một chiều trong đó mỗi dòng của mảng là một mảng một chiều, và truy xuất từng phần tử bằng cách truy xuất bởi chỉ số dòng và cột.

Khai báo

```
Kiểu dữ liệu Tên_mảng[số dòng tối đa][số cột tối đa];
```

Ví dụ:

```
int A[10][10]; // khai báo mảng 2 chiều kiểu int gồm 10 dòng, 10 cột  
float B[10][10]; // khai báo mảng 2 chiều kiểu float có 10 dòng, 10  
cột
```

Truy cập

```
Tên_mảng[chỉ số dòng][chỉ số cột];
```

Ma trận vuông và các khái niệm liên quan

Khái niệm: là ma trận có số dòng và số cột bằng nhau. Trong đó:

- Đường chéo chính có **chỉ số dòng = chỉ số cột**.
- Đường chéo phụ có: **chỉ số cột + chỉ số dòng = số dòng (hoặc số cột)**.

6.2 THỰC HÀNH CƠ BẢN

Câu 1: Viết chương trình gồm các hàm sau:

- Nhập ma trận gồm d dòng và c cột (d, c nhập từ bàn phím)
- Xuất ma trận
- Tính tổng các phần tử của ma trận
- Tính trung bình cộng các phần tử trong ma trận
- Tính trung bình cộng các phần tử dương trong ma trận
- Xuất các phần tử nằm trên dòng **k** (**k** do người dùng nhập) trong ma trận
- Tính tổng các phần tử nằm trên cột **k** (**k** do người dùng nhập) trong ma trận
- Tìm phần tử lớn nhất trong ma trận.

Hướng dẫn:

```
#include<stdio.h>

#define MAX 10
typedef int MATRAN[MAX][MAX];

//b1: nhập số dòng và số cột của ma trận
void Nhap(int &d, int &c)
{
    printf("Nhap so dong va so cot:");
    scanf("%d%d",&d,&c);
}

//b2: nhập giá trị cho từng phần tử của ma trận
void NhapMang2C(MATRAN a, int d, int c)
{
    for(int i=0; i<d; i++){
        for(int j=0; j<c; j++){
            printf("a[%d][%d]= ",i,j);
            scanf("%d",&a[i][j]);
        }
    }
}
```

```

    }
}
//b3: xuất từng phần tử của ma trận
void XuatMang2C(MATRAN a, int d, int c)
{
    for(int i=0; i<d; i++){
        for(int j=0; j<c; j++){
            printf("%d \t ",a[i][j]);
        }
        printf("\n");
    }
}

```

Câu 2: Ma trận vuông cấp n là mảng 2 chiều có (số dòng) = (số cột) = n .

- Sinh ngẫu nhiên 1 ma trận vuông cấp n chứa số nguyên (n nhập từ bàn phím).
- Xuất ma trận
- Liệt kê các phần tử trên đường chéo chính của ma trận
- Liệt kê các phần tử trên đường chéo phụ của ma trận
- Tính tổng các phần tử nằm trên dòng k (k do người dùng nhập)
- Tính tổng các phần tử trên mỗi dòng của ma trận
- Xuất các giá trị cùng với tổng của dòng có tổng lớn nhất.

Hướng dẫn:

a. Sinh ngẫu nhiên 1 ma trận vuông

```

void NhapMang2C(MATRAN a, int n){
    srand((int)time(0));
    for(int i=0; i<n; i++){
        for(int j=0; j<n; j++){
            {
                a[i][j] = 1 + rand() % (10);
            }
        }
    }
}

```

```

    }
}

```

c. Liệt kê các phần tử trên đường chéo chính của ma trận

Điều kiện để đường chéo chính là dòng bằng cột (hay $i=j$) khi duyệt i theo dòng và duyệt j theo cột.

d. Liệt kê các phần tử trên đường chéo phụ của ma trận

Điều kiện để đường chéo phụ ($j = n-i-1$) khi duyệt i theo dòng và duyệt j theo cột.

e. Tính tổng các phần tử nằm trên dòng k

```

long TongDongK(MATRAN a, int n){
    int k;
    long tong=0;
    //sinh viên tự nhập dòng thứ k ( $0 \leq k < n$ )
    for(int j=0; j<n; j++)
    {
        tong += a[k][j];
        printf("%d \t ", a[k][j]);
    }
    return tong;
}

```

Câu 3: Viết chương trình nhập ma trận vuông kích thước $n \times n$ ($2 \leq n \leq 100$).

Hãy viết hàm thực hiện những công việc sau:

- In ra các phần tử trên 4 đường biên của ma trận
- Tính tổng các phần tử của ma trận trừ các phần tử nằm trên 4 đường biên của ma trận
- Kiểm tra ma trận vuông có đối xứng qua đường chéo chính hay không
- Tính tổng các phần tử nằm trên đường chéo chính và chéo phụ của ma trận. (Trường hợp với n lẻ thì trừ phần tử bị lặp lại tại đường giao của hai đường chéo trong ma trận).

6.3 THỰC HÀNH NÂNG CAO

Câu 4: Viết chương trình in ra tam giác Pascal.

1	Thuật toán:			
1 1	Dòng 1 chứa 1 số; dòng 2 chứa 2 số, ..., dòng n chứa n số			
1 2 1	Số hạng đầu và số hạng cuối là 1.			
1 3 3 1	Số hạng ở giữa bằng tổng 2 số hạng của dòng trên cộng lại			
1 4 6 4 1				
...				

Câu 5: Viết chương trình tính tổng của hai ma trận các số nguyên.

Hướng dẫn:

- Khai báo 3 ma trận: `int matran1[10][10], matran2[10][10], matran3[10][10]`
- Nhập ma trận thứ 1: `matran1`
- Nhập ma trận thứ 2: `matran2`
- Tổng hai ma trận: `matran3`

```
//duyet vòng for i theo dòng
//duyet vòng for j theo cột
// phần tử của ma trận 3 = phần tử của ma trận 1 + phần tử của ma trận 2
for (i = 0; i < dong; i++)
    for (j = 0; j < cot; j++) {
        matran3[i][j] = matran1[i][j] + matran2[i][j];
    }
```

6.4 BÀI TẬP VỀ NHÀ

Câu 6: Hãy viết hàm thực hiện những công việc sau:

- Nhập số dòng và số cột của ma trận
- Sinh ma trận ngẫu nhiên

Gợi ý: sử dụng thư viện `#include <time.h>` và `#include <stdlib.h>`

`srand((int)time(0))`

`minN + rand() % (maxN + 1 - minN)`

- c. Xuất ma trận vừa sinh ngẫu nhiên
- d. Tính tổng biên của ma trận
- e. Tính trung bình cộng các phần tử của ma trận
- f. Kiểm tra ma trận vừa rồi có phải ma trận vuông hay không? Nếu ma trận vuông thì tính tổng hai đường chéo của ma trận
- g. Sắp xếp ma trận tăng dần theo dòng
- h. Sắp xếp các phần tử của ma trận giảm dần đều.

BÀI 7: CHUỖI KÝ TỰ

Sau khi học xong bài này, sinh viên có thể nắm được:

- Khai báo và sử dụng kiểu dữ liệu chuỗi ký tự;
- Các thao tác nhập xuất, các kỹ thuật thao tác trên chuỗi ký tự.

7.1 TÓM TẮT LÝ THUYẾT

Chuỗi là một dãy ký tự dùng để lưu trữ và xử lý văn bản như từ, cụm từ, câu. Trong ngôn ngữ C kiểu chuỗi được thể hiện bằng mảng các ký tự (có kiểu cơ sở char) và kết thúc bằng ký tự '\0' (ký tự NULL trong bảng mã ASCII).

Hằng chuỗi ký tự được đặt trong cặp dấu nháy kép "".

Khai báo

```
char <tên-biến> [chiều dài tối đa chuỗi];  
char <tên-biến> [] = <"Hằng chuỗi">;
```

Ví dụ: char Hoten [20];

Ví dụ trên khai báo 1 chuỗi chứa tối đa 19 ký tự (1 ký tự cuối của chuỗi chứa NULL)

```
char cslt[ ] = "Cơ sở lập trình";
```

Khai báo và khởi tạo biến chuỗi cslt chứa giá trị khởi tạo là "Cơ sở lập trình".

Nhập xuất trên chuỗi

Nhập chuỗi: **gets(biến-chuỗi);** // nhập từ bàn phím cho đến khi nhấn Enter

Sử dụng hàm **fflush(stdin)** để xóa vùng đệm trước khi nhập

Hoặc dùng hàm: scanf("%s", &biến-chuỗi); // hàm scanf nhập chuỗi

không có khoảng trắng.

Xuất chuỗi: **puts (biến-chuỗi);** //Xuất chuỗi xong tự động xuống dòng

7.2 THỰC HÀNH CƠ BẢN

Câu 1: Viết chương trình nhập và xuất hai chuỗi

Hướng dẫn:

```
#include <stdio.h>
#include <string.h>
#define MAX_LEN 100

int main() {
    char s1[MAX_LEN], s2[MAX_LEN];
    int len1, len2;
    scanf("%s", s1);
    len1 = strlen(s1);
    scanf("%s", s2);
    len2 = strlen(s2);
    printf("%s %d\n", s1, len1);
    printf("%s %d", s2, len2);
    return 0;
}
```

Câu 2: Viết chương trình so sánh hai chuỗi.

Hướng dẫn:

```
#include <stdio.h>
#include <string.h>
#define MAX_LEN 100
int main() {
    char s1[MAX_LEN], s2[MAX_LEN];
    int len1, len2;
    scanf("%s", s1);
```

```
scanf("%s", s2);
int result = strcasecmp(s1, s2);
if (result < 0)
{
    printf("<");
}
else if (result > 0)
{
    printf(">");
}
else
{
    printf("=");
}
return 0;
}
```

Câu 3: Viết chương trình nối hai chuỗi.

Hướng dẫn:

```
#include <stdio.h>
#include <string.h>
#define MAX_LEN 100

int main() {
    char s1[MAX_LEN], s2[MAX_LEN];
    gets(s1);
    gets(s2);
    strcat(s1, " ");
    strcat(s1, s2);
    printf("%s", s1);
    return 0;
}
```

Câu 4: Viết chương trình kiểm tra chuỗi có chứa chuỗi con.

Hướng dẫn:

```
#include <stdio.h>
#include <string.h>
#define MAX_LEN 100
int main() {
    char s1[MAX_LEN], s2[MAX_LEN];
    gets(s1);
    gets(s2);
    if (strstr(s1, s2) != NULL)
    {
        printf("YES");
    }
    else
    {
        printf("NO");
    }
    return 0;
}
```

Câu 5: Viết chương trình kiểm tra tính đối xứng của chuỗi vừa nhập.

Câu 6: Viết chương trình đếm số lần xuất hiện của **ký tự** x trong chuỗi.

Câu 7: Viết chương trình đếm số lần xuất hiện của **từ** y trong chuỗi.

Câu 8: Viết chương trình sao chép chuỗi này sang một chuỗi khác.

Câu 9: Viết chương trình để trích xuất một chuỗi con từ một chuỗi đã cho.

7.3 THỰC HÀNH NÂNG CAO

Câu 10: Viết chương trình chuyển chuỗi s nhập từ bàn phím thành chuỗi chữ viết hoa, chuỗi viết thường và chuỗi viết hoa mỗi chữ cái đầu mỗi từ.

Câu 11: Viết chương trình đếm tổng số chữ cái, chữ số và ký tự đặc biệt trong một chuỗi.

Câu 12: Viết chương trình tìm số lần xuất hiện của các ký tự trong chuỗi.

Câu 13: Viết chương trình tìm từ dài nhất và ngắn nhất trong một chuỗi.

Câu 14: Viết chương trình kiểm tra một ký tự có phải là chữ số hay không.

7.4 BÀI TẬP VỀ NHÀ

Viết hàm thực hiện các yêu cầu sau

- Tìm chuỗi con Palindromic dài nhất từ một chuỗi đã cho.
- Thay thế từng chữ viết thường bằng cùng một chữ cái viết hoa của một chuỗi đã cho.
- Xóa tất cả các ký tự trong chuỗi ngoại trừ các ký tự trong bảng chữ cái.
- Đếm số ký tự là nguyên âm (a, e, i, o, u) và phụ âm trong một chuỗi.
- Đếm số lần xuất hiện nhiều nhất của một ký tự trong chuỗi.
- Kiểm tra chuỗi có chứa hoàn toàn các ký tự in hoa, ký tự in thường.

TÀI LIỆU THAM KHẢO

- [1]Phạm Văn Ất, Kỹ thuật Lập trình C- cơ bản và nâng cao, NXB KH & KT - 2003.
- [2]Nguyễn Tấn Trần Minh Khang, Bài giảngKỹ Thuật Lập trình, Khoa Công Nghệ Thông Tin, Đại học Khoa học Tự nhiên
- [3]Nguyễn Thanh Thủy (chủ biên), Nhập môn lập trình ngôn ngữ C, Nhà xuất bản Khoa học kỹ thuật – 2000.
- [4]Trần Minh Thái, Bài giảng và bài tập Lập trình căn bản; Khoa Công Nghệ Thông Tin, Đại học Khoa học Tự nhiên
- [5]Trần Minh Thái, Giáo Trình Bài Tập Kỹ Thuật Lập Trình, Cao đẳng Công Nghệ Thông Tin Tp. Hồ Chí Minh
- [6]Brain W. Kernighan & Dennis Ritchie, The C Programming Language, Prentice Hall Publisher, 1988.